Data Provenance Metadata Version 1.0

Committee Specification Draft 02

19 July 2026

This version

- https://docs.oasis-open.org/dps/prov-meta/v1.0/csd02/prov-meta-v1.0-csd02.md (Authoritative)
- https://docs.oasis-open.org/dps/prov-meta/v1.0/csd02/prov-meta-v1.0-csd02.html
- https://docs.oasis-open.org/dps/prov-meta/v1.0/csd02/prov-meta-v1.0-csd02.pdf

Latest version

- https://docs.oasis-open.org/dps/prov-meta/v1.0/prov-meta-v1.0.md (Authoritative)
- https://docs.oasis-open.org/dps/prov-meta/v1.0/prov-meta-v1.0.html
- https://docs.oasis-open.org/dps/prov-meta/v1.0/prov-meta-v1.0.pdf

Previous version

- https://docs.oasis-open.org/dps/prov-meta/v1.0/csd01/prov-meta-v1.0-csd01.md (Authoritative)
- https://docs.oasis-open.org/dps/prov-meta/v1.0/csd01/prov-meta-v1.0-csd01.html
- https://docs.oasis-open.org/dps/prov-meta/v1.0/csd01/prov-meta-v1.0-csd01.pdf

Technical Committee

Data Provenance Standard Technical Committee

Chairs

- Lisa Bobbitt, Cisco, lbobbitt@cisco.com
- Bryan Bortnick, IBM, bortnick@us.ibm.com
- Fotis Psallidas, Microsoft, Fotis.Psallidas@microsoft.com

Secretaries

- Jamie Yu, Cisco, jamyu2@cisco.com

Editors

- David Kemp, NSA, d.kemp@cyber.nsa.gov
- Stefan Hagen, individual, stefan@hagen.link

Additional Artifacts

This prose specification is one component of a Work Product that also includes:

- Data Provenance Metadata JSON schema: https://docs.oasis-open.org/dps/prov-meta/v1.0/csd02/schema/data-provenance.json.
  Latest stage: https://docs.oasis-open.org/dps/prov-meta/v1.0/schema/data-provenance.json.
- Data Provenance Metadata Configuration JSON schema: https://docs.oasis-open.org/dps/prov-meta/v1.0/csd02/schema/data-provenance-configuration.json.
  Latest stage: https://docs.oasis-open.org/dps/prov-meta/v1.0/schema/data-provenance-configuration.json.

Declared JSON namespaces

- https://docs.oasis-open.org/dps/prov-meta/v1.0/schema/data-provenance.json
- https://docs.oasis-open.org/dps/prov-meta/v1.0/schema/data-provenance-configuration.json

Abstract

This OASIS Data Provenance Metadata specification provides an information model and several specialized data schemata for describing and managing data provenance and data lineage. The resulting common language provides transparency for

data provenance and enables assessing where data comes from, how it has been created, and in what scenarios it can be used, legally.

Citation Format

When referencing this document, the following citation format should be used:

[prov-meta-v1.0]

Data Provenance Metadata Version 1.0. Edited by David Kemp and Stefan Hagen. 19 July 2026. OASIS Committee Specification Draft 02.
https://docs.oasis-open.org/dps/prov-meta/v1.0/csd02/prov-meta-v1.0-csd02.html.
Latest stage: https://docs.oasis-open.org/dps/prov-meta/v1.0/prov-meta-v1.0.html.

Related Work

This document replaces or supersedes:

N/A

This document is related to:

N/A

License, Document Status, and Notices

Table of Contents

1. Scope

Data is a core enterprise asset that underpins strategic decision-making, drives operational priorities, and supports risk governance. Dependence on data creates a need for validation by understanding data's origin, quality, and intended use. Understanding data is a requirement for organizations operating at scale. The OASIS Data Provenance Standards (DPS) are created to solve for this need. Developed through cross-industry collaboration, the DPS provide a consistent framework to track the origin, movement, integrity, and quality of data. The DPS address the growing demand for transparency in artificial intelligence (AI), cybersecurity, supply chains, and areas where data quality and accountability are foundational to performance and compliance - especially in regulated and high-risk environments.

2. Definitions and Acronyms

2.1 Definitions

2.1.1 Terms Defined Elsewhere

This document uses the following terms defined elsewhere:

Data Provenance
: In the context of computers and law enforcement use, it is synonymous to chain of custody. It involves the method of generation, transmission and storage of information that may be used to trace the origin of a piece of information processed by community resources. Cf. [CNSSI-4009] for more details.

Data Lineage
: Data lineage is the process of tracking the (use and) flow of data over time, providing a clear understanding of where the data originated, how it has changed, and its ultimate destination within the data pipeline. Cf. "Data Lineage at IBM" https://www.ibm.com/think/topics/data-lineage for more details.

Data Transparency
: Data transparency refers to the clear, open, and honest handling of data within an organization. It means that businesses, governments, and institutions disclose how they collect, store, use, and share data, ensuring users, customers, and stakeholders understand their practices. Cf. "What is data transparency at BigID" https://bigid.com/blog/what-is-data-transparency for more details.

2.1.2 Terms Defined in this Document

None

2.2 Abbreviations and Acronyms

AI
: Artificial Intelligence

DET
: Data Enhancing Technologies

DPS
: Data Provenance Standard

PET
: Privacy Enhancing Technologies

3. Document Conventions

3.1 Key Words

The key words "**MUST**", "**MUST NOT**", "**REQUIRED**", "**SHALL**", "**SHALL NOT**", "**SHOULD**", "**SHOULD NOT**", "**RECOMMENDED**", "**NOT RECOMMENDED**", "**MAY**", and "**OPTIONAL**" in this document are to be interpreted as described in BCP 14 [RFC2119] and [RFC8174] when, and only when, they appear in all capitals, as shown here.

3.2 Typographical Conventions

Keywords defined by this specification use this `monospaced` font.

```
Normative source code uses this paragraph style.
```

Some sections of this specification are illustrated with non-normative examples introduced with "Example" or "Examples" like so:

Example 1:

```
Informative examples also use this paragraph style but preceded by the text "Example(s)".
```

All examples in this document are informative only.

All other text is normative unless otherwise labeled e.g. like the following informative comment:

> This is a pure informative comment that may be present, because the information conveyed is deemed useful advice or common pitfalls learned from implementer or operator experience and often given including the rationale.

This document adheres to the Modern Language Association (MLA) style guidelines for formatting titles and terms.

4. Introduction

Data is a core enterprise asset. It underpins strategic decision-making, drives operational priorities, and supports risk governance. Dependence on data creates a need for validation and an understanding of the data's origin, quality, and intended use. Understanding data is a requirement for organizations operating at scale. The OASIS Data Provenance Standards (DPS) are created to solve for this need.

4.1 Any Additional Introduction Subsections That are Needed

None

4.2 Changes From the Previous Version

N/A

5. Provenance Schema

The schema of the provenance metadata is described in human-readable property tables. The technical encoding may be found in section 6 "Provenance Information Model Encoding".

The Data Provenance Standards record metadata elements in three segmented categories: Source, Provenance, and Use.

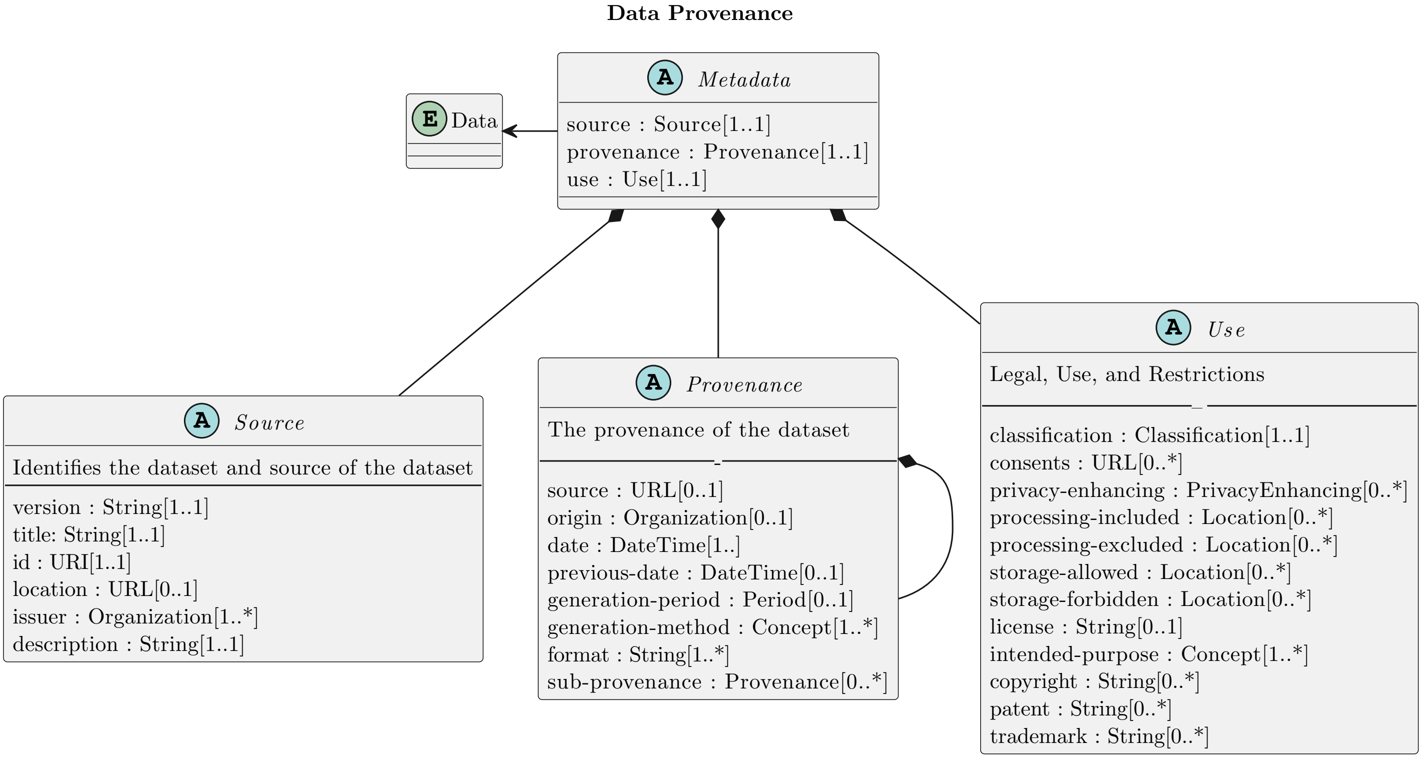

Figure 1: Metadata

The property tables first define metadata about the specification itself, then describe how a record is made of the 3 primary metadata elements. The three segmented categories (Source, Provenance, and Use) are comprised of various metadata element input fields. Each field is described in more detail below.

5.1 Primary Metadata Elements

The root object of a Data Provenance Metadata record contains five required members.

Table 1: Type DataProvenance (Record)

ID	Name	Type	#	Description
1	$schema	String /uri	1	The URI identifying the schema this JSON object must be valid against. For this version the value is always the URI of the data-provenance JSON schema.
2	set	Set	1	Set-level metadata about this provenance metadata record.
3	source	Source	1	Characterizes the content and source of the dataset.
4	provenance	Provenance	1	Describes the provenance of the dataset.
5	use	Use	1	Describes legal use and restrictions that apply to the dataset.

5.2 Set

The set member captures metadata about the provenance metadata record itself.

Table 2: Type Set (Record)

ID	Name	Type	#	Description
1	category	String	1	A short canonical name, chosen by the set producer, informing the end user of the category of the metadata set.
2	schema-version	String	1	The version of the Data Provenance Core specification this record was generated for. The value for this version is always "1.0".
3	publisher	Publisher	1	Information about the publisher of the metadata set.
4	label	String	1	The per-publisher unique title of this metadata set.
5	tracking	Tracking	1	Management attributes used to track this metadata set as a whole.
6	language	Language	0..1	The language used in this document, corresponding to IETF BCP 47.
7	source-language	Language	0..1	If this is a translation, the language this document was translated from.
8	acknowledgments	Acknowledgment	0..*	Acknowledgment elements associated with the whole set.
9	notes	Note	0..*	Notes associated with this metadata set.
10	references	Reference	0..*	References associated with this metadata set.

Table 3: Type Publisher (Record)

ID	Name	Type	#	Description
1	name	String	1	The name of the issuing party.
2	namespace	String /uri	1	A URI under the control of the issuing party, used as a globally unique identifier for that party.
3	contact-details	String	0..1	Information on how to contact the publisher.

Table 4: Type Tracking (Record)

ID	Name	Type	#	Description
1	current-release-date	String /date-time	1	The date when the current revision of this document was released.
2	id	String	1	A unique identifier for this metadata set, assigned and maintained by the original issuing authority. MUST NOT start or end with whitespace.
3	initial-release-date	String /date-time	1	The date when this metadata set was first published.
4	revision-history	Revision	1..*	One revision item for each version of the metadata set, including the initial one.
5	status	TrackingStatus	1	The draft status of the metadata set.
6	version	Version	1	The version of the current metadata set.
7	aliases	String	0..*	Alternate names for this metadata set.
8	generator	Generator	0..1	Information about the engine that generated this metadata set.

Table 5: Type TrackingStatus (Enumerated)

ID	Item	Description
0	draft	The metadata set is a draft.
1	final	The metadata set is final.

ID	Item	Description
2	interim	The metadata set has interim status.

Table 6: Type Revision (Record)

ID	Name	Type	#	Description
1	date	String /date-time	1	The date of this revision entry.
2	number	Version	1	The version number of this revision.
3	summary	String	1	A short description of the changes in this revision.
4	legacy-version	String	0..1	A version string from an existing set with the same content, tracing from DP-Core compliant to historic items.

Table 7: Type Generator (Record)

ID	Name	Type	#	Description
1	engine	GeneratorEngine	1	Information about the engine used to generate this metadata set.
2	date	String /date-time	0..1	The date this metadata set was generated.

Table 8: Type GeneratorEngine (Record)

ID	Name	Type	#	Description
1	name	String	1	The name of the engine that generated this metadata set.
2	version	Version	0..1	The version of the engine that generated this metadata set.

Table 9: Type Note (Record)

ID	Name	Type	#	Description
1	category	NoteCategory	1	The category of the note.
2	text	String	1	The content of the note, varying depending on type.
3	audience	String	0..1	The intended audience for the note.
4	title	String	0..1	A concise description of what is contained in the text of the note.

Table 10: Type NoteCategory (Enumerated)

ID	Item	Description
0	description	A descriptive note about the subject.
1	details	A note with detailed supplemental information.
2	faq	A note in frequently-asked-questions format.
3	general	A general note.
4	legal-disclaimer	A note containing a legal disclaimer.
5	other	A note that does not fit other categories.
6	summary	A summary note.

Table 11: Type Reference (Record)

ID	Name	Type	#	Description
1	summary	String	1	A short description of what this reference refers to.
2	url	String /uri	1	The URL of the referenced resource.
3	category	ReferenceCategory	0..1	Whether this reference is external or self-referential.

Table 12: Type ReferenceCategory (Enumerated)

ID	Item	Description
0	external	The reference points to an external resource.
1	self	The reference points back to the same information item in focus.

5.3 Source

The source member characterizes the content and source of the dataset.

Table 13: Type Source (Record)

ID	Name	Type	#	Description
1	about	About	1	A detailed narrative explaining the contents, scope, and purpose of the dataset.
2	id	Identity	1	A unique identifier for this dataset, using at least one identification method.
3	issuer	Organization	1..*	The legal entity or entities responsible for creating the dataset.
4	location	String /uri	1	The web address where this dataset's metadata is published and can be accessed.
5	name	String	1	The official name of the dataset.
6	data-version	Version	1	The version of the dataset this metadata set describes.

Table 14: Type About (Record)

ID	Name	Type	#	Description
1	content	String	1	Contextual information helping users understand what the data represents and how it was collected.
2	purpose	String	1	The recommended uses for this dataset.
3	scope	String	0..1	Any limitations of the dataset.

The Identity type provides at least one identifier for the dataset. At least one of its fields MUST be present.

Table 15: Type Identity (Map)

ID	Name	Type	#	Description
1	hashes	HashEntry	0..*	Cryptographic hash-based identifiers for the dataset.
2	uris	String /uri	0..*	URI identifiers for the dataset.
3	uuids	String /uuid	0..*	UUID identifiers for the dataset.
4	custom-ids	CustomId	0..*	Identifiers in any text format.

Table 16: Type HashEntry (Record)

ID	Name	Type	#	Description
1	tree-hashes	TreeHash	1..*	The cryptographic hash values for this file or tree.

ID	Name	Type	#	Description
2	path	String	1	The path (file or filesystem tree root) identified by these hash values.

Table 17: Type TreeHash (Record)

ID	Name	Type	#	Description
1	algorithm	String	1	The name of the hash or tree/seal algorithm used to calculate the value.
2	value	String	1	The cryptographic hash value in hexadecimal representation.

Table 18: Type CustomId (Record)

ID	Name	Type	#	Description
1	method	String	1	The name of the method used to derive this identifier.
2	value	String	1	The identifier value.
3	tool	String	0..1	The name of the tool used to derive this identifier.
4	parameter-list	String	0..*	Parameters used when invoking the tool.

5.4 Provenance

The provenance member describes the provenance of the dataset.

Table 19: Type Provenance (Record)

ID	Name	Type	#	Description
1	origin-geography	GeographicRegion	1..*	The geographical locations where the data was originally collected.
2	date	String /date	1	The date when the dataset was compiled or created (YYYY-MM-DD).
3	generation-method	Generation-Method	1..*	The methodology or procedures used to collect, generate, or compile the data.
4	source	String /uri	0..1	The URI of metadata for a source dataset contributing to this dataset, establishing lineage.
5	origin	Organization	0..*	The original source organization, if different from the dataset issuer.
6	previous-date	String /date	0..1	The release date of the previous version of this dataset (YYYY-MM-DD).
7	generation-period	GenerationPeriod	0..1	The span of time during which the data was collected or generated.
8	format	ModalityFormat	0..*	Describes the modality or media type of the data within the dataset.
9	sub-provenance	Provenance	0..1	Nested provenance information for a component of this dataset.

Table 20: Type GenerationPeriod (Record)

ID	Name	Type	#	Description
1	start	String /date-time	0..1	The start of the data collection or generation period.
2	end	String /date-time	0..1	The end of the data collection or generation period.

Table 21: Type GenerationMethod (Record)

ID	Name	Type	#	Description
1	code	Generation-MethodCode	1	A short code identifying the generation method.
2	system	String	0..1	The code system used to interpret the code value.
3	long-description	String	0..1	A detailed description of the method.

Table 22: Type GenerationMethodCode (Enumerated)

ID	Item	Description
0	data-augmentation	Data generation through augmentation of existing data.
1	data-mining	Data collected through automated data mining.
2	feeds	Data received through data feeds.
3	machine-generated-ml-ops	Data generated through machine learning operations.
4	other	A generation method not listed here.
5	primary-user-source	Data collected directly from primary users.
6	simulations	Data generated through simulations.
7	social-media	Data collected from social media platforms.
8	syndication	Data received through syndication.
9	transfer-learning	Data generated through transfer learning.
10	user-generated-content	Data created by end users.
11	web-scraping-crawling	Data collected by web scraping or crawling.

Table 23: Type ModalityFormat (Enumerated)

ID	Item	Description
0	application/json	JSON data format.
1	application/jsonld	JSON-LD linked data format.
2	application/msword	Microsoft Word document format.
3	application/	Microsoft Excel spreadsheet format.

ID	Item	Description
	vnd.ms-excel	
4	application/zip	ZIP archive format.
5	image/bmp	BMP image format.
6	image/gif	GIF image format.
7	image/jpeg	JPEG image format.
8	image/png	PNG image format.
9	image/x-png	PNG image format (legacy media type).
10	other	A format not listed here.
11	text/csv	CSV text format.
12	text/plain	Plain text format.

5.5 Use

The use member describes the legal use and restrictions applicable to the dataset.

Table 24: Type Use (Record)

ID	Name	Type	#	Description
1	intended-purpose	Purpose	1..*	The purpose or purposes for which the dataset was created.
2	classification	Classification	0..*	Confidentiality classifications applicable to the dataset.
3	consents	String	0..*	Locations of consent documentation or agreements related to the data.
4	data-risk-reducing	DataRiskReducing	0..*	Techniques used to reduce known data risk.
5	processing-included	GeographicRegion	0..*	Geographical boundaries within which the data may be processed.
6	processing-excluded	GeographicRegion	0..*	Geographical boundaries within which the data may not be processed.
7	storage-allowed	GeographicRegion	0..*	Geographical locations where the data may be stored.
8	storage-forbidden	GeographicRegion	0..*	Geographical locations where the data may not be stored.
9	license	String	0..*	The terms under which the dataset can be used, including any restrictions or obligations.
10	copyright	String	0..*	Copyright information covering proprietary information in the dataset.
11	patent	String	0..*	Patent information covering proprietary information in the dataset.
12	trademark	String	0..*	Trademark information covering proprietary information in the dataset.

Table 25: Type Purpose (Record)

ID	Name	Type	#	Description
1	code	PurposeCode	1	A short code identifying the intended purpose.
2	long-description	String	1	A detailed description of the intended purpose.
3	system	String	0..1	The code system used to interpret the code value.

Table 26: Type Classification (Record)

ID	Name	Type	#	Description
1	regulation	Regulation	1	The regulation or classification standard being evaluated.
2	evaluated	Boolean	1	Whether the dataset has been evaluated against this classification (true) or not (false).
3	tool	String	0..1	The tool used to perform the evaluation.

Table 27: Type Regulation (Record)

ID	Name	Type	#	Description
1	code	ConfidentialityCode	1	A short code identifying the confidentiality classification.
2	system	String	0..1	The code system used to interpret the code value.
3	long-description	String	0..1	A detailed description of the regulation or classification.

Table 28: Type ConfidentialityCode (Enumerated)

ID	Item	Description
0	other	A confidentiality classification not listed here.
1	pci	Payment Card Industry (PCI) data.
2	pfi	Personal Financial Information (PFI).
3	phi	Protected Health Information (PHI).
4	pi	Personal Information (PI).
5	sci	Sensitive Compartmented Information (SCI).
6	spi	Sensitive Personal Information (SPI).

Table 29: Type PurposeCode (Enumerated)

ID	Item	Description
0	alignment	Dataset intended for model alignment tasks.
1	evaluation	Dataset intended for evaluation or benchmarking.
2	other	A purpose not listed here.
3	pre-training	Dataset intended for model pre-training.
4	production	Dataset intended for production use.
5	quality-assurance	Dataset intended for quality assurance testing.
6	research	Dataset intended for research use.
7	staging-testing	Dataset intended for staging or testing environments.

ID	Item	Description
8	synthetic-data-generation	Dataset intended for generating synthetic data.

Table 30: Type DataRiskReducing (Record)

ID	Name	Type	#	Description
1	tool-used	String	1	The name of the tool used for data risk reduction.
2	tool-category	Method	0..1	The category of the tool used for data risk reduction.
3	technology	DataTechnology	0..1	The privacy-enhancing or data-reducing technology applied.
4	parameters	Parameter	0..*	The parameters passed to the tool.
5	results	String	0..*	The results of applying the tool.

Table 31: Type Parameter (Record)

ID	Name	Type	#	Description
1	name	String	1	The parameter name.
2	value	String	0..1	The parameter value.

Table 32: Type DataTechnology (Enumerated)

ID	Item	Description
0	data-anonymization	Transforms the dataset so that individuals cannot be identified, directly or indirectly, from the resulting records.
1	data-encryption	Encodes dataset content so that it is readable only by parties holding the decryption key; typically applied at rest or in transit.
2	data-masking	Replaces sensitive field values with realistic but fictitious substitutes, preserving format and statistical properties without exposing the originals.
3	data-minimization	Reduces the dataset to only the fields and records necessary for the stated purpose, limiting exposure of personal or sensitive data.
4	data-redaction	Removes or obscures specific values or text spans identified as sensitive, such as names, identifiers, or health information.
5	differential-privacy	Adds calibrated statistical noise so that the presence or absence of any individual record cannot be reliably inferred from the released data.
6	federated-learning	Produces model artefacts by training across distributed sources without centralising raw data, so the original dataset is never directly shared.
7	homomorphic-encryption	Enables computation on ciphertext without decryption, so analytical results can be derived without exposing plaintext values.
8	k-anonymity	Generalises quasi-identifying attributes so that each record is indistinguishable from at least k-1 other records in the dataset.
9	l-diversity	Extends k-anonymity by requiring that each equivalence class contains at least l well-represented distinct sensitive attribute values.
10	other	A data risk reduction technology not covered by the defined values; use tool-category or results to describe the specific technique applied.

ID	Item	Description
11	pseudo-nymization	Replaces direct identifiers with artificial pseudonyms; re-identification remains possible through a separate, controlled mapping table.
12	secure-multi-party-computation	Allows multiple parties to jointly compute over their respective inputs without revealing those inputs to each other (SMC).
13	t-closeness	Extends l-diversity by requiring that the distribution of sensitive values within each equivalence class closely matches their overall distribution in the dataset.
14	tokenization	Substitutes sensitive values with non-sensitive tokens; the originals are recoverable only through a separate, secured token vault.

5.6 Shared Types

The following types are used in more than one of the sections above.

Table 33: Type Organization (Record)

ID	Name	Type	#	Description
1	legal-name	String	1	The legal name of the organization.
2	address	Address	0..1	The postal address of the organization.
3	url	String /uri	0..1	A URL for the organization.

Table 34: Type Address (ArrayOf(String))

Type Name	Type Definition	Description
Address	ArrayOf(String)	Lines of a postal address. At least one required.

Table 35: Type GeographicRegion (Record)

ID	Name	Type	#	Description
1	country	String	1	The country name or code.
2	state	String	0..1	The state or province name or code.

Table 36: Type Method (Record)

ID	Name	Type	#	Description
1	code	String	1	A short code identifying the method, regulation, or classification.
2	system	String	0..1	The code system used to interpret the code value.
3	long-description	String	0..1	A detailed description of the method, regulation, or classification.

The Acknowledgment type describes a contributor. At least one of its fields MUST be present.

Table 37: Type Acknowledgment (Map)

ID	Name	Type	#	Description
1	names	String	0..*	Names of individual contributors being acknowledged.
2	organization	String	0..1	Name of a contributing organization being acknowledged.
3	summary	String	0..1	Contextual details about the acknowledgment.

ID	Name	Type	#	Description
4	urls	String /uri	0..*	URLs of references being acknowledged.

Table 38: Type Version (String)

Type Name	Type Definition	Description
Version	String	An integer or semantic versioning string. Examples: "1", "2.0.0", "1.0.0-beta".

Table 39: Type Language (String)

Type Name	Type Definition	Description
Language	String	A language tag conforming to IETF BCP 47 / RFC 5646.

6. Provenance Information Model Encoding

The technical encoding of the information model is specified in both JADN and YAML in the following subsections.

6.1 JADN Encoding

The JADN encoding of the data provenance metadata information model is specified in `jadn/data-provenance.jadn`.

The file contains 43 named types derived directly from the property tables in Section 5. The exported root type is `DataProvenance`. Six named primitive aliases (`URI`, `UUID`, `DateTime`, `Date`, `Version`, `Language`) carry format constraints that apply wherever those types appear as field types, keeping the field-level definitions free of inline format options. Eight enumerated types carry the closed code vocabularies introduced in Section 5.

6.2 YAML Encoding

The YAML encoding of the data provenance metadata information model is specified in `yaml/data-provenance.yaml`.

This file is a YAML serialization of `json/data-provenance.json` and is normatively equivalent to it. It is provided for tooling that consumes YAML-format schemas natively. All field names, constraints, type references, and enumeration values are identical to those in the JSON schema.

7. Provenance Data Model Encoding

The information model defines the complete set of metadata elements and associated attributes specified by the Data Provenance Standard. It establishes a common conceptual framework for representing provenance information, including the structures and relationships necessary to describe the origin, history, and handling of data. The information model is intended to provide a consistent semantic basis for provenance across implementations, independent of any particular serialization or storage mechanism.

In order to support interoperability and exchange of provenance information between systems, the information model requires one or more concrete encodings. An encoding provides a standardized, machine-readable representation of the information model suitable for electronic transmission, persistence, and processing. While the information model defines what information is conveyed, the encoding defines how that information is represented for exchange between conforming implementations.

This section describes a set of possible encodings for the Data Provenance Standard. Each encoding maps the constructs defined in the information model to a specific representation format intended for storage or system-to-system exchange. The encodings described herein are non-exclusive and are provided to support diverse implementation environments and usage scenarios. Implementations MAY support one or more of these encodings, subject to their interoperability, performance, and deployment requirements.

7.1 JSON Encoding

The technical encoding of the data provenance metadata data model is specified in the following schema artifacts for JSON data:

- Data Provenance Metadata JSON schema
- Data Provenance Metadata Configuration JSON schema

The Data Provenance Metadata Configuration JSON schema configures validators to enforce all type and subtype constraints.

The Data Provenance Metadata JSON schema provides definitions and the structure for any data provenance metadata JSON instance.

The required top-level members are:

```
DataProvenance:
  $schema: String.Constant
  set: Mapping
  source: Mapping
  provenance: Mapping
  use: Mapping
```

7.1.1 Member `$schema`

The value of the `$schema` member for this version of the specification is always:

```
https://docs.oasis-open.org/dps/prov-meta/v1.0/schema/data-provenance.json
```

7.1.2 Member `set`

The `set` member specifies set-level meta-data. It captures meta-data about the provenance metadata record describing a particular dataset.

The required members of `set` are `category`, `schema-version`, `publisher`, `label`, and `tracking`. The optional members `language`, `source-language`, `acknowledgments`, `notes`, and `references` MAY also be present.

```
DataProvenance:
  # ...
  set:
    category: String.Pattern
    schema-version: String.Constant
    publisher: Mapping
    label: String
    tracking: Mapping
```



```
    # language: String.BCP47         (optional)
    # source-language: String.BCP47 (optional)
    # acknowledgments: Sequence      (optional)
    # notes: Sequence                (optional)
    # references: Sequence           (optional)
  # ...
# ...
```

7.1.2.1 Member `set.category`

The `set.category` member defines a short canonical name, chosen by the set producer, which will inform the end user as to the category of the metadata set.

```
DataProvenance:
  # ...
  set:
    category: String.Pattern
    # ...
  # ...
# ...
```

Examples 1:

```
dp_base
dp_event_source
dp_profile_xyz
Example Data Protection Notice Exemption
```

7.1.2.2 Member `set.schema-version`

The `set.schema-version` member gives the version of the Data Provenance Core specification which the document was generated for.

```
DataProvenance:
  # ...
  set:
    # ...
    schema-version: String.Constant
    # ...
  # ...
# ...
```

The value of the `set.schema-version` member for this version of the specification is always:

```
1.0
```

7.1.2.3 Member `set.publisher`

The `set.publisher` member provides information about the publisher of the metadata set. The publisher is the party responsible for issuing the metadata set.

The required members of `set.publisher` are `name` and `namespace`. The optional member `contact-details` MAY also be present.

```
DataProvenance:
  # ...
  set:
    # ...
    publisher:
      name: String
      namespace: String.URI
      # contact-details: String   (optional)
    # ...
  # ...
# ...
```

7.1.2.3.1 Member `set.publisher.name`

The `set.publisher.name` member contains the name of the issuing party.

```
DataProvenance:
  # ...
  set:
```



```
    # ...
    publisher:
      name: String
      # ...
    # ...
  # ...
# ...
```

Example 1:

```
Cisco
```

7.1.2.3.2 Member `set.publisher.namespace`

The `set.publisher.namespace` member contains a URI which is under the control of the issuing party and can be used as a globally unique identifier for that issuing party.

```
DataProvenance:
  # ...
  set:
    # ...
    publisher:
      # ...
      namespace: String.URI
      # ...
    # ...
  # ...
# ...
```

The value of `set.publisher.namespace` MUST be a valid URI.

Example 1:

```
https://cisco.com
```

7.1.2.3.3 Member `set.publisher.contact-details`

The `set.publisher.contact-details` member provides information on how to contact the publisher, such as web sites, email addresses, phone numbers, or postal mail addresses.

```
DataProvenance:
  # ...
  set:
    # ...
    publisher:
      # ...
      contact-details: String
    # ...
  # ...
# ...
```

Example 1:

```
Example Company can be reached at contact_us@example.com, or via our website at https://www.example.com/contact.
```

7.1.2.4 Member `set.label`

The `set.label` member provides the per-publisher unique title of the metadata set.

```
DataProvenance:
  # ...
  set:
    # ...
    label: String
    # ...
  # ...
# ...
```

The value of `set.label` SHOULD be a canonical name for the set, and sufficiently unique to distinguish it from similar sets.

Examples 1:


```
Learning Set for Regression Modelling for Stats 101
Example Data Protection Dataset in Example Generator
```

7.1.2.5 Member `set.tracking`

The `set.tracking` member is a container designated to hold all management attributes necessary to track a DP-Core set as a whole.

The required members of `set.tracking` are `current-release-date`, `id`, `initial-release-date`, `revision-history`, `status`, and `version`. The optional members `aliases` and `generator` MAY also be present.

```yaml
DataProvenance:
  # ...
  set:
    # ...
    tracking:
      current-release-date: String.DateTime
      id: String.Pattern
      initial-release-date: String.DateTime
      revision-history: Sequence
      status: String.Enum
      version: $defs.version-type
      # aliases: Sequence   (optional)
      # generator: Mapping  (optional)
    # ...
  # ...
# ...
```

7.1.2.5.1 Member `set.tracking.current-release-date`

The `set.tracking.current-release-date` member contains the date when the current revision of this document was released.

```yaml
DataProvenance:
  # ...
  set:
    # ...
    tracking:
      current-release-date: String.DateTime
      # ...
    # ...
  # ...
# ...
```

The value of `set.tracking.current-release-date` MUST be a valid date-time string conforming to [RFC 3339].

Example 1:

```
2000-01-01T01:01:01Z
```

7.1.2.5.2 Member `set.tracking.id`

The `set.tracking.id` member provides the unique identifier for the metadata set. The ID is a simple label that provides for a wide range of numbering values, types, and schemes. Its value SHOULD be assigned and maintained by the original metadata set issuing authority.

```yaml
DataProvenance:
  # ...
  set:
    # ...
    tracking:
      # ...
      id: String.Pattern
      # ...
    # ...
  # ...
# ...
```

The value of `set.tracking.id` MUST NOT start or end with whitespace.

Examples 1:


```
7aedeb0a-22dd-428a-ab76-c950b43cbbc6
abcdef-orga-ds-0815
cisco-sa-20190513-secureboot
```

7.1.2.5.3 Member `set.tracking.initial-release-date`

The `set.tracking.initial-release-date` member contains the date when this set was first published.

```
DataProvenance:
  # ...
  set:
    # ...
    tracking:
      # ...
      initial-release-date: String.DateTime
      # ...
    # ...
  # ...
# ...
```

The value of `set.tracking.initial-release-date` MUST be a valid date-time string conforming to [RFC 3339].

Example 1:

```
2000-01-01T01:01:01Z
```

7.1.2.5.4 Member `set.tracking.revision-history`

The `set.tracking.revision-history` member holds one revision item for each version of the DP-Core set, including the initial one. The sequence MUST contain at least one revision item.

```
DataProvenance:
  # ...
  set:
    # ...
    tracking:
      # ...
      revision-history:
        - date: String.DateTime
          number: $defs.version-type
          summary: String
          # legacy-version: String  (optional)
      # ...
    # ...
  # ...
# ...
```

Each revision item MUST contain the following members:

- `date`: The date of the revision entry. MUST be a valid date-time string conforming to [RFC 3339].
- `number`: The version number of the revision. MUST conform to `$defs.version-type` (integer or semantic versioning string).
- `summary`: A short description of the changes made in this revision.

The optional `legacy-version` member traces entries back to historic, DP-Core-unaware version strings.

Example 1:

```
[
  {
    "date": "2000-01-01T01:01:01Z",
    "number": "1",
    "summary": "Initial version."
  }
]
```

7.1.2.5.5 Member `set.tracking.status`

The `set.tracking.status` member defines the draft status of the metadata set. This allows processing DP-Core sets of various maturity per version.


```yaml
DataProvenance:
  # ...
  set:
    # ...
    tracking:
      # ...
      status: String.Enum
      # ...
    # ...
  # ...
# ...
```

The value of `set.tracking.status` MUST be one of the following:

- draft
- final
- interim

7.1.2.5.6 Member `set.tracking.version`

The `set.tracking.version` member specifies the version of the current DP-Core set.

```yaml
DataProvenance:
  # ...
  set:
    # ...
    tracking:
      # ...
      version: $defs.version-type
      # ...
    # ...
  # ...
# ...
```

The value of `set.tracking.version` MUST conform to `$defs.version-type`, which requires either an integer or a semantic versioning string.

Examples 1:

```
1
2.0.0
1.0.0-beta+exp.sha.a1c44f85
```

7.1.2.5.7 Member `set.tracking.aliases`

The `set.tracking.aliases` member contains a list of alternate names for the same set. Each alias MUST be a non-empty string and all entries MUST be unique.

```yaml
DataProvenance:
  # ...
  set:
    # ...
    tracking:
      # ...
      aliases:
        - String
    # ...
  # ...
# ...
```

Example 1:

```
["0bacba95-ceb9-4fae-bf07-d5683a9481c1"]
```

7.1.2.5.8 Member `set.tracking.generator`

The `set.tracking.generator` member holds information about the engine that generated the metadata set. The required sub-member is `engine`. The optional sub-member `date` MAY also be present.

```yaml
DataProvenance:
  # ...
  set:
```



```
    # ...
    tracking:
      # ...
      generator:
        engine:
          name: String
          # version: $defs.version-type   (optional)
        # date: String.DateTime   (optional)
    # ...
  # ...
# ...
```

- engine.name: The name of the tool or system that produced the metadata set.
- engine.version: The version of that tool. MUST conform to $defs.version-type when present.
- date: The date-time at which the metadata set was generated. MUST conform to [RFC 3339] when present.

Example 1:

```
{
  "engine": {"name": "DPCoreX", "version": "1.0.0"},
  "date": "2025-01-15T10:00:00Z"
}
```

7.1.2.6 Member set.language

The set.language member identifies the language used in this document, corresponding to IETF BCP 47 / RFC 5646.

```
DataProvenance:
  # ...
  set:
    # ...
    language: String.BCP47
    # ...
  # ...
# ...
```

Examples 1:

```
de
en
fr
```

7.1.2.7 Member set.source-language

The set.source-language member identifies the language this document was translated from, when this copy is a translation.

```
DataProvenance:
  # ...
  set:
    # ...
    source-language: String.BCP47
    # ...
  # ...
# ...
```

7.1.2.8 Member set.acknowledgments

The set.acknowledgments member contains a list of acknowledgment elements associated with the whole set. Each acknowledgment item MUST have at least one of the following members: names, organization, summary, or urls.

```
DataProvenance:
  # ...
  set:
    # ...
    acknowledgments:
      - # names: Sequence        (optional)
        # organization: String   (optional)
        # summary: String        (optional)
        # urls: Sequence         (optional)
  # ...
# ...
```


Example 1:

```
[
  {
    "names": ["A. Big Data"],
    "organization": "ETH Zurich",
    "summary": "First analysis of the merged data-set"
  }
]
```

7.1.2.9 Member `set.notes`

The `set.notes` member contains a list of notes associated with this metadata set. Each note MUST have a `category` and `text`. The optional members `audience` and `title` MAY also be present. The `category` value MUST be one of: `general`, `other`, `privacy`, `summary`.

```
DataProvenance:
  # ...
  set:
    # ...
    notes:
      - category: String.Enum  # NoteCategory: description | details | faq | general | legal_disclaimer | other | summary
        text: String
        # audience: String  (optional)
        # title: String     (optional)
  # ...
# ...
```

- `category`: The category of the note.
- `text`: The content of the note, varying depending on type.
- `audience`: The intended audience for the note.
- `title`: A concise description of what is contained in the text of the note.

7.1.2.10 Member `set.references`

The `set.references` member contains a list of references associated with this metadata set. Each reference MUST have a `summary` and a `url`. The optional member `category` MAY also be present. The `category` value MUST be one of: `external`, `self`.

```
DataProvenance:
  # ...
  set:
    # ...
    references:
      - summary: String
        url: String.URI
        # category: String.Enum  # ReferenceCategory: external | self (optional)
  # ...
# ...
```

- `summary`: A short description of what this reference refers to.
- `url`: The URL of the referenced resource.
- `category`: Whether this reference is external or self-referential (default: `external`).

7.1.3 Member `source`

The `source` member characterizes the content and source of the dataset. All six of its members are required.

```
DataProvenance:
  # ...
  source:
    about: Mapping
    id: Mapping
    issuer: Sequence
    location: String.URI
    name: String
    data-version: $defs.version-type
  # ...
# ...
```

The members of `source` are described in the following subsections.

7.1.3.1 Member `source.about`

The `source.about` member contains a detailed narrative that explains the contents, scope, and purpose of the dataset. The required members are `content` and `purpose`. The optional member `scope` MAY also be present.

```
DataProvenance:
  # ...
  source:
    # ...
    about:
      content: String
      purpose: String
      # scope: String  (optional)
    # ...
  # ...
# ...
```

- `content`: Provides essential contextual information that helps users understand what the data represents and how it was collected.
- `purpose`: Explains the recommended uses.
- `scope`: Points out any limitations of the dataset.

Example 1:

```
{
  "content": "We found these numbers on the parking lot.",
  "purpose": "Use only for learning regression modeling. Not for production use.",
  "scope": "We did not verify anything. Use at your own risk."
}
```

7.1.3.2 Member `source.id`

The `source.id` member provides a unique identifier for the dataset's metadata. At least one identification method MUST be provided.

```
DataProvenance:
  # ...
  source:
    # ...
    id:  # at least one of the following
      hashes: Sequence
      uris: Sequence
      uuids: Sequence
      custom-ids: Sequence
    # ...
  # ...
# ...
```

The following identification methods are available:

- `hashes`: A list of cryptographic hash entries, each containing `tree-hashes` (a list of `{algorithm, value}` pairs) and a `path`.
- `uris`: A list of identifiers in URI format.
- `uuids`: A list of identifiers in UUID format.
- `custom-ids`: A list of identifiers in any text format, each with a required `method` and `value`, and optional `tool` and `parameter-list`.

Example 1:

```
{
  "uuids": ["e5471657-9ede-4335-843b-c1376ef29bfa"]
}
```

Example 2:

```
{
  "hashes": [
    {
      "tree-hashes": [{"algorithm": "sha256", "value": "af1349b9f5f9a1a6a0404dea36dcc9499bcb25c9adc112b7cc9a93cae41f3262"}],
      "path": "example-data.tar.xz"
    }
  ]
}
```


7.1.3.3 Member `source.issuer`

The `source.issuer` member identifies the legal entity or entities responsible for creating the dataset. The value is a sequence of organization objects. Each organization MUST provide a `legal-name`. The sequence MUST contain at least one organization and all entries MUST be unique. The optional members `address` and `url` MAY also be present in each organization.

```
DataProvenance:
  # ...
  source:
    # ...
    issuer:
      - legal-name: String
        # address: Sequence  (optional)
        # url: String.URI    (optional)
    # ...
  # ...
# ...
```

Example 1:

```
[
  {
    "legal-name": "Sampling Ltd.",
    "url": "https://example.com/smplng/about"
  }
]
```

7.1.3.4 Member `source.location`

The `source.location` member provides the web address where the dataset's metadata is published and can be accessed. Typically this will be a unique URI of the current dataset.

```
DataProvenance:
  # ...
  source:
    # ...
    location: String.URI
    # ...
  # ...
# ...
```

7.1.3.5 Member `source.name`

The `source.name` member provides the official name of the dataset, which should be descriptive and help easily identify the dataset's content and purpose.

```
DataProvenance:
  # ...
  source:
    # ...
    name: String
    # ...
  # ...
# ...
```

7.1.3.6 Member `source.data-version`

The `source.data-version` member specifies the version of the dataset this DP-Core set describes, allowing the dataset to evolve over time and keeping consistent labeling.

```
DataProvenance:
  # ...
  source:
    # ...
    data-version: $defs.version-type
    # ...
  # ...
# ...
```

The value of `source.data-version` MUST conform to `$defs.version-type`, which requires either an integer or a semantic versioning string.

Examples 1:


```
1
2.0.0
```

7.1.4 Member `provenance`

The `provenance` member describes the provenance of the dataset.

The required members are `origin-geography`, `date`, and `generation-method`. The optional members `source`, `origin`, `previous-date`, `generation-period`, `format`, and `sub-provenance` MAY also be present.

```
DataProvenance:
  # ...
  provenance:
    origin-geography: Sequence
    date: String.Date
    generation-method: Sequence
    # source: String.URI           (optional)
    # origin: Sequence             (optional)
    # previous-date: String.Date    (optional)
    # generation-period: Mapping    (optional)
    # format: Sequence              (optional)
    # sub-provenance: Mapping       (optional, recursive)
  # ...
# ...
```

The required members of `provenance` are described in subsections 7.1.4.1–7.1.4.3; optional members in 7.1.4.4–7.1.4.9.

7.1.4.1 Member `provenance.origin-geography`

The `provenance.origin-geography` member identifies the geographical locations where the data was originally collected, which can be important for compliance with regional laws and understanding the data's context.

The value is a sequence of geographic region objects. Each entry MUST contain a `country`. The optional `state` member MAY also be present. The sequence MUST contain at least one entry and all entries MUST be unique.

```
DataProvenance:
  # ...
  provenance:
    # ...
    origin-geography:
      - country: String
        # state: String  (optional)
    # ...
  # ...
# ...
```

Example 1:

```
[
  {"country": "US"},
  {"country": "DE", "state": "Bavaria"}
]
```

7.1.4.2 Member `provenance.date`

The `provenance.date` member provides the date when the dataset was compiled or created, providing a temporal context for the data.

```
DataProvenance:
  # ...
  provenance:
    # ...
    date: String.Date
    # ...
  # ...
# ...
```

The value of `provenance.date` MUST be a valid date string in full-date format (YYYY-MM-DD).

Example 1:


```
2000-01-01
```

7.1.4.3 Member `provenance.generation-method`

The `provenance.generation-method` member describes the methodology or procedures used to collect, generate, or compile the data, giving insight into its reliability and validity.

The value is a sequence of method objects. Each method MUST contain a `code` drawn from the `GenerationMethodCode` vocabulary. The optional members `system` and `long-description` MAY also be present. The sequence MUST contain at least one method and all entries MUST be unique.

```
DataProvenance:
  # ...
  provenance:
    # ...
    generation-method:
      - code: String.Enum  # GenerationMethodCode: data-augmentation | data-mining | feeds | machine-generated-ml-ops | other | primary-user-source | simulations | social-media | syndication | transfer-learning | user-generated-content | web-scraping-crawling
        # system: String           (optional)
        # long-description: String (optional)
    # ...
  # ...
# ...
```

Example 1:

```
[
  {"code": "web-scraping-crawling"},
  {"code": "primary-user-source", "long-description": "Direct survey responses from participants."}
]
```

7.1.4.4 Member `provenance.source`

The `provenance.source` member identifies where the metadata for the source dataset contributing to the current dataset can be found, establishing lineage.

```
DataProvenance:
  # ...
  provenance:
    # ...
    source: String.URI
    # ...
  # ...
# ...
```

The value of `provenance.source` MUST be a valid URI.

Example 1:

```
https://example.com/datasets/source-dataset-42/provenance.json
```

7.1.4.5 Member `provenance.origin`

The `provenance.origin` member identifies the original source organization when it differs from the issuer of the dataset. The value is a sequence of organization objects following the same structure as `source.issuer`.

```
DataProvenance:
  # ...
  provenance:
    # ...
    origin:
      - legal-name: String
        # address: Sequence  (optional)
        # url: String.URI    (optional)
    # ...
  # ...
# ...
```

Example 1:


```
[{"legal-name": "Original Data Collector Inc."}]
```

7.1.4.6 Member `provenance.previous-date`

The `provenance.previous-date` member provides the release date of the previous version of the dataset, to track changes and updates over time.

```
DataProvenance:
  # ...
  provenance:
    # ...
    previous-date: String.Date
    # ...
  # ...
# ...
```

The value of `provenance.previous-date` MUST be a valid date string in full-date format (YYYY-MM-DD).

Example 1:

```
1999-06-15
```

7.1.4.7 Member `provenance.generation-period`

The `provenance.generation-period` member describes the span of time during which the data within the dataset was collected or generated. Both `start` and `end` are optional but MUST each conform to [RFC 3339] when present.

```
DataProvenance:
  # ...
  provenance:
    # ...
    generation-period:
      # start: String.DateTime  (optional)
      # end: String.DateTime    (optional)
    # ...
  # ...
# ...
```

Example 1:

```
{
  "start": "1998-01-01T00:00:00Z",
  "end": "1999-12-31T23:59:59Z"
}
```

7.1.4.8 Member `provenance.format`

The `provenance.format` member describes the modality or media type of the data within the dataset. The value is a sequence of `ModalityFormat` values. The sequence MUST contain at least one entry and all entries MUST be unique.

```
DataProvenance:
  # ...
  provenance:
    # ...
    format:
      - String.Enum  # ModalityFormat: application/json | application/jsonld | application/msword | application/vnd.ms-excel | application/zip | image/bmp | image/gif | image/jpeg | image/png | image/x-png | other | text/csv | text/plain
    # ...
  # ...
# ...
```

Example 1:

```
["text/csv", "application/json"]
```

7.1.4.9 Member `provenance.sub-provenance`

The `provenance.sub-provenance` member holds provenance information for a component of the current dataset, enabling a recursive provenance tree. Its structure is identical to the enclosing `provenance` member.


```yaml
DataProvenance:
  # ...
  provenance:
    # ...
    sub-provenance:
      origin-geography: Sequence
      date: String.Date
      generation-method: Sequence
      # ... (same optional members as provenance)
    # ...
  # ...
# ...
```

7.1.5 Member use

The use member describes legal use and restrictions that apply to the dataset.

The only required member is intended-purpose. All other members are optional.

```yaml
DataProvenance:
  # ...
  use:
    intended-purpose: Sequence
    # classification: Sequence       (optional)
    # consents: Sequence             (optional)
    # data-risk-reducing: Sequence   (optional)
    # processing-included: Sequence (optional)
    # processing-excluded: Sequence (optional)
    # storage-allowed: Sequence      (optional)
    # storage-forbidden: Sequence    (optional)
    # license: Sequence              (optional)
    # copyright: Sequence            (optional)
    # patent: Sequence               (optional)
    # trademark: Sequence            (optional)
  # ...
# ...
```

7.1.5.1 Member use.intended-purpose

The use.intended-purpose member describes the purpose for which the dataset was created, guiding users on its intended use and potential applications.

The value is a sequence of purpose objects. Each purpose MUST contain a code drawn from the PurposeCode vocabulary and a long-description. The optional system member MAY also be present. The sequence MUST contain at least one purpose and all entries MUST be unique.

```yaml
DataProvenance:
  # ...
  use:
    # ...
    intended-purpose:
      - code: String.Enum  # PurposeCode: alignment | evaluation | other | pre-training | production | quality-assurance | research | staging-testing | synthetic-data-generation
        long-description: String
        # system: String  (optional)
    # ...
  # ...
# ...
```

Example 1:

```json
[
  {
    "code": "research",
    "long-description": "Use only for learning regression modeling. Not for production use."
  }
]
```

7.1.5.2 Member use.classification

The use.classification member indicates whether the dataset includes data falling into various confidentiality classifications. The value is a sequence of classification objects. Each item MUST contain a regulation object whose code MUST be drawn from the ConfidentialityCode vocabulary, and an evaluated boolean. The optional tool member MAY also be present. All entries MUST be unique.


```
DataProvenance:
  # ...
  use:
    # ...
    classification:
      - regulation:
          code: String.Enum  # ConfidentialityCode: other | pci | pfi | phi | pi | sci | spi
          # system: String           (optional)
          # long-description: String (optional)
        evaluated: Boolean
        # tool: String  (optional)
    # ...
  # ...
# ...
```

Example 1:

```
[
  {"regulation": {"code": "phi"}, "evaluated": true},
  {"regulation": {"code": "pii"}, "evaluated": false}
]
```

7.1.5.3 Member `use.consents`

The `use.consents` member specifies where consent documentation or agreements related to the data can be found, ensuring legal compliance and regulatory use. The value is a sequence of strings. The sequence MUST contain at least one entry and all entries MUST be unique.

```
DataProvenance:
  # ...
  use:
    # ...
    consents:
      - String
    # ...
  # ...
# ...
```

Example 1:

```
["https://example.com/consent-policy-2024"]
```

7.1.5.4 Member `use.data-risk-reducing`

The `use.data-risk-reducing` member indicates the techniques used to reduce known data risk. The value is a sequence of objects. Each item MUST contain `tool-used`. The optional members `tool-category`, `technology`, `parameters`, and `results` MAY also be present. All entries MUST be unique.

```
DataProvenance:
  # ...
  use:
    # ...
    data-risk-reducing:
      - tool-used: String
        # tool-category: Mapping  (optional, method object)
        # technology: String.Enum (optional, DataTechnology value)
        # parameters: Sequence    (optional, list of {name, value} pairs)
        # results: Sequence       (optional, list of strings)
    # ...
  # ...
# ...
```

Example 1:

```
[
  {
    "tool-used": "DataMaskPro",
    "technology": "data-masking",
    "parameters": [{"name": "--level", "value": "high"}],
    "results": ["All PII fields masked."]
```



```
  }
]
```

7.1.5.5 Member `use.processing-included`

The `use.processing-included` member defines the geographical boundaries within which the data MAY be processed. The value is a sequence of geographic region objects following the same structure as `provenance.origin-geography`.

```yaml
DataProvenance:
  # ...
  use:
    # ...
    processing-included:
      - country: String
        # state: String  (optional)
    # ...
  # ...
# ...
```

Example 1:

```json
[{"country": "DE"}, {"country": "FR"}]
```

7.1.5.6 Member `use.processing-excluded`

The `use.processing-excluded` member defines the geographical boundaries within which the data MUST NOT be processed. The value is a sequence of geographic region objects following the same structure as `provenance.origin-geography`.

```yaml
DataProvenance:
  # ...
  use:
    # ...
    processing-excluded:
      - country: String
        # state: String  (optional)
    # ...
  # ...
# ...
```

7.1.5.7 Member `use.storage-allowed`

The `use.storage-allowed` member specifies geographical locations where the data MAY be stored. The value is a sequence of geographic region objects following the same structure as `provenance.origin-geography`.

```yaml
DataProvenance:
  # ...
  use:
    # ...
    storage-allowed:
      - country: String
        # state: String  (optional)
    # ...
  # ...
# ...
```

7.1.5.8 Member `use.storage-forbidden`

The `use.storage-forbidden` member specifies geographical locations where the data MUST NOT be stored. The value is a sequence of geographic region objects following the same structure as `provenance.origin-geography`.

```yaml
DataProvenance:
  # ...
  use:
    # ...
    storage-forbidden:
      - country: String
        # state: String  (optional)
    # ...
  # ...
# ...
```


7.1.5.9 Member `use.license`

The `use.license` member details the terms under which the dataset can be used, including any restrictions or obligations. License entries MAY include End User License Agreements (EULA) or Data Use Agreements (DUA). The value is a sequence of strings. The sequence MUST contain at least one entry and all entries MUST be unique.

```
DataProvenance:
  # ...
  use:
    # ...
    license:
      - String
    # ...
  # ...
# ...
```

Example 1:

```
["https://creativecommons.org/licenses/by/4.0/"]
```

7.1.5.10 Member `use.copyright`

The `use.copyright` member indicates whether the dataset contains proprietary information covered by a copyright, and the terms of that copyright. The value is a sequence of strings. The sequence MUST contain at least one entry and all entries MUST be unique.

```
DataProvenance:
  # ...
  use:
    # ...
    copyright:
      - String
    # ...
  # ...
# ...
```

Example 1:

```
["Copyright © 2024 Example Corp. All rights reserved."]
```

7.1.5.11 Member `use.patent`

The `use.patent` member indicates whether the dataset contains proprietary information covered by a patent, and the patent number. The value is a sequence of strings. The sequence MUST contain at least one entry and all entries MUST be unique.

```
DataProvenance:
  # ...
  use:
    # ...
    patent:
      - String
    # ...
  # ...
# ...
```

7.1.5.12 Member `use.trademark`

The `use.trademark` member indicates whether the dataset contains proprietary information covered by a trademark, and the terms of that trademark. The value is a sequence of strings. The sequence MUST contain at least one entry and all entries MUST be unique.

```
DataProvenance:
  # ...
  use:
    # ...
    trademark:
      - String
    # ...
  # ...
# ...
```


8. Safety, Security, and Data Protection

All safety, security, and data protection requirements relevant to the context in which Data Provenance Metadata documents are used MUST be translated into, and consistently enforced through, Data Provenance Metadata implementations and processes.

For Data Provenance Metadata documents based on JSON, the security considerations of [RFC8259] apply and are repeated here as a service for the reader:

> Generally, there are security issues with scripting languages. JSON is a subset of JavaScript but excludes assignment and invocation.
>
> Since JSON's syntax is borrowed from JavaScript, it is possible to use that language's `eval()` function to parse most JSON texts (but not all; certain characters such as U+2028 LINE SEPARATOR and U+2029 PARAGRAPH SEPARATOR are legal in JSON but not JavaScript). This generally constitutes an unacceptable security risk, since the text could contain executable code along with data declarations. The same consideration applies to the use of eval()-like functions in any other programming language in which JSON texts conform to that language's syntax.

Implementations SHOULD validate Data Provenance Metadata documents against the normative JSON Schema prior to processing. Documents that do not validate MUST NOT be treated as authoritative provenance records.

8.1 Integrity and Authenticity of Provenance Claims

This specification defines the structure and vocabulary of provenance metadata but does not define mechanisms for verifying that the claims made within a record are accurate or have not been tampered with. A Data Provenance Metadata document is only as trustworthy as the process and party that produced it.

Implementations that rely on provenance metadata for access control, compliance, or risk decisions SHOULD obtain records through channels that provide integrity guarantees, such as transport-layer security (TLS) when fetching from the URI identified in `source.location`, or cryptographic signatures applied to the document by the `set.publisher`.

The `set.tracking.id` and `source.id` fields provide identifiers for the record and the described dataset respectively, but do not themselves provide integrity protection.

8.2 URI Safety

Several fields in this specification accept URI values, including `source.location`, `set.publisher.namespace`, `source.id.uris`, `set.acknowledgments[].urls`, and `set.references[].url`. Implementations that automatically dereference these URIs MUST apply the same defences required for any externally supplied URL, including protection against server-side request forgery (SSRF) and open redirect attacks. URIs MUST be validated as well-formed before dereferencing. Implementations SHOULD restrict dereferenceable schemes to `https` and, where appropriate, `http`.

8.3 Sensitive Information in Provenance Records

Provenance records may incidentally contain information that publishers consider sensitive, including internal tool names in `use.data-risk-reducing`, personnel names in `set.acknowledgments`, internal geographic constraints in `use.processing-included` and related fields, and organizational contact details in `set.publisher.contact-details`.

Publishers MUST consider whether information included in a provenance record is appropriate for the audience that will consume it. Provenance records intended for public distribution SHOULD be reviewed to ensure they do not expose internal operational details beyond what is required to describe the dataset's provenance.

8.4 Data Protection Compliance

The `use.classification` field records whether a dataset has been evaluated against confidentiality classifications such as PHI, PI, PCI, SCI, or SPI. The presence of a classification entry, including the value of the `evaluated` field, does not itself constitute compliance with any applicable law or regulation. Implementers that process datasets described by provenance records containing such classifications MUST apply the data handling controls required by the relevant regulations in the jurisdictions where the data is processed and stored.

The geographic restriction fields (use.processing-included, use.processing-excluded, use.storage-allowed, use.storage-forbidden) express the publisher's stated intent but are not self-enforcing. Implementers are responsible for establishing technical and organizational controls that honour these restrictions.

9. Conformance

This document defines requirements for the data-provenance in the JSON file format and for certain software components that interact with it.

Annex A. License, Document Status and Notices

(This annex forms an integral part of this Specification.)

A.1 Document Status

This document was last revised or approved by the OASIS DPS TC on the above date. The level of approval is also listed above. Check the "Latest version" location noted above for possible later revisions of this document. Any other numbered Versions and other technical work produced by the Technical Committee (TC) are listed at https://groups.oasis-open.org/communities/tc-community-home2?CommunityKey=2c60b2cf-45d3-48cd-8594-0194f182b33d.

TC members should send comments on this document to the TC's email list. Others should send comments to the TC's public comment list, after subscribing to it by following the instructions at the "Send A Comment" button on the TC's web page at https://www.oasis-open.org/committees/dps/.

NOTE: any machine-readable content (Computer Language Definitions) declared Normative for this Work Product is provided in separate plain text files. In the event of a discrepancy between any such plain text file and display content in the Work Product's prose narrative document(s), the content in the separate plain text file prevails.

A.2 License and Notices

Annex B. References

(This annex forms an integral part of this Specification.)

This section contains the normative and informative references that are used in this document.

Normative references are specific (identified by date of publication and/or edition number or version number) and Informative references are either specific or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the reference document (including any amendments) applies. While any hyperlinks included in this section were valid at the time of publication, OASIS cannot guarantee their long term validity.

B.1 Normative References

The following documents are referenced in such a way that some or all of their content constitutes requirements of this document.

[RFC2119] *Key Words for Use in RFCs to Indicate Requirement Levels*, BCP 14, RFC 2119, March 1997. [Online]. Available: https://www.rfc-editor.org/info/rfc2119

[RFC8174] *Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words*, BCP 14, RFC 8174, May 2017. [Online]. Available: https://www.rfc-editor.org/info/rfc8174

[RFC8259] Bray, T., "The JavaScript Object Notation (JSON) Data Interchange Format", RFC 8259, DOI 10.17487/RFC8259, December 2017, http://www.rfc-editor.org/info/rfc8259.

B.2 Informative References

The following referenced documents are not required for the application of this document but may assist the reader with regard to a particular subject area.

[CNSSI-4009] *Committee on National Security Systems (CNSS) Glossary*, Committee on National Security Systems, CNSSI No. 4009, April 6, 2015, Available: https://nsarchive.gwu.edu/document/22385-document-08-committee-national-security

Annex C. Example Instances

(This annex forms an integral part of this Specification.)

This annex provides three example instances of the Data Provenance Metadata schema, each serving a distinct purpose in learning and applying the standard.

The **minimal example** (C.1 "Minimal Example") contains only the fields that are required by the schema, populated with the smallest valid values. It is intended as a starting point for new implementers and as a quick reference for the mandatory structure of a DP-Core set.

The **moderate example** (C.2 "Moderate Example") extends the minimal case by adding a representative selection of optional fields at each level. It is intended to illustrate common real-world usage — a dataset with multiple geographies, a revision history, licensing information, and a confidentiality classification — without attempting to cover every possible field.

The **complete example** (C.3 "Complete Example") is a comprehensive instance that exercises every field defined in the schema at least once. It is not intended to represent a typical implementation; its purpose is to serve as a normative reference for tool developers verifying parser completeness and for authors checking field semantics under realistic conditions.

All three instances are available as JSON files in the `examples/` directory of the specification distribution.

C.1 Minimal Example

The following instance contains only the fields required by the schema. The scenario is a single-issuer survey dataset with one generation method and one intended purpose.

```json
{
  "$schema": "https://docs.oasis-open.org/dps/prov-meta/v1.0/schema/data-provenance.json",
  "set": {
    "category": "training-data",
    "schema-version": "1.0",
    "publisher": {
      "name": "Example Organization",
      "namespace": "https://example.org/"
    },
    "label": "Example Dataset Provenance",
    "tracking": {
      "current-release-date": "2026-06-30T00:00:00Z",
      "id": "example-dataset-prov-001",
      "initial-release-date": "2026-06-30T00:00:00Z",
      "revision-history": [
        {
          "date": "2026-06-30T00:00:00Z",
          "number": "1",
          "summary": "Initial release."
        }
      ],
      "status": "final",
      "version": "1"
    }
  },
  "source": {
    "about": {
      "content": "A dataset of anonymized customer survey responses collected in 2025.",
      "purpose": "Intended for training binary sentiment analysis models."
    },
    "id": {
      "uuids": ["f47ac10b-58cc-4372-a567-0e02b2c3d479"]
    },
    "issuer": [
      {
        "legal-name": "Example Organization"
      }
    ],
    "location": "https://example.org/datasets/survey-2025/provenance.json",
    "name": "Customer Survey Dataset 2025",
    "data-version": "1"
  },
  "provenance": {
    "origin-geography": [
      {
        "country": "United States"
      }
    ],
    "date": "2026-06-01",
```



```
    "generation-method": [
      {
        "code": "primary-user-source"
      }
    ]
  },
  "use": {
    "intended-purpose": [
      {
        "code": "pre-training",
        "long-description": "To be used for pre-training binary sentiment classification models."
      }
    ]
  }
}
```

The file `examples/dp-core-minimal.json` contains this instance.

C.2 Moderate Example

The moderate example extends the minimal case with a representative but non-exhaustive selection of optional fields. The scenario is a multilingual news corpus assembled from syndicated feeds and published by a single commercial data provider.

Key features illustrated by this instance:

- **Set level**: `language`, `acknowledgments`, and publisher `contact-details` are present. The tracking block includes two revision entries, reflecting a real update cycle.
- **Source level**: `scope` is populated in `about`, limiting the stated purpose of the dataset. Two identification methods are used (`uuids` and `uris`). The issuer entry includes a postal `address` and a `url`.
- **Provenance level**: Three `origin-geography` entries span multiple countries. `generation-method` carries a `long-description` explaining the feed mechanism. `format`, `generation-period`, and a lineage `source` URI are all present.
- **Use level**: Two `intended-purpose` entries distinguish pre-training from academic research use. A single `classification` entry records evaluation against the Personal Information (PI) regulation. `consents` and `license` are populated.

The file `examples/dp-core-moderate.json` contains this instance.

C.3 Complete Example

The complete example is a comprehensive instance designed to exercise every field in the schema. The scenario is a large merged biomedical text corpus assembled by a research consortium from three distinct sub-sources: PubMed abstracts, ClinicalTrials.gov result summaries, and de-identified hospital discharge summaries.

Key features illustrated by this instance that are not present in the moderate example:

- **Set level**: All optional tracking fields are used, including `aliases` and `generator` (with engine name, version, and generation date). `source-language`, `notes`, and `references` are present. The `notes` array contains two entries — a `legal_disclaimer` for the clinical content and a `summary` note describing what changed in version 3. The `references` array demonstrates both `external` and `self` category values.
- **Source level**: All four `identity` methods are used: `uuids`, `uris`, `hashes` (with a SHA-256 tree hash), and `custom-ids` (a DataCite DOI with tool parameters). Two `issuer` entries reflect the dual institutional origin. `scope` in `about` explicitly excludes document types not covered.
- **Provenance level**: Three `generation-method` entries with different codes cover the full collection pipeline. `format` lists three media types. `origin` names the upstream data providers. `previous-date` establishes continuity with the prior version. `sub-provenance` records the provenance of the initial PubMed-only sub-corpus from the v1 release.
- **Use level**: Three `intended-purpose` entries (`pre-training`, `alignment`, `evaluation`). Three `classification` entries cover PHI, PI, and a free-text GDPR Article 9 entry using `code: other` with a `long-description`. Two `data-risk-reducing` entries document the NER-based redaction tool and the k-anonymity pipeline, each with `technology`, `parameters`, and `results`. All four geography restriction fields are populated (`processing-included`, `processing-excluded`, `storage-allowed`, `storage-forbidden`). `copyright`, `patent`, and `trademark` are present.

The file `examples/dp-core-complete.json` contains this instance.

Appendix A. Acknowledgments

(This appendix does not form an integral part of this Specification and is informational.)

The following individuals were members of the OASIS DPS Technical Committee during the creation of this specification and their contributions are gratefully acknowledged:

Leadership

The following individuals have had significant leadership positions during the development of this document, not just this version of the document, and they are gratefully acknowledged:

- Chairs
  - Lisa Bobbitt, Cisco, lbobbitt@cisco.com
  - Bryan Bortnick, IBM, bortnick@us.ibm.com
  - Fotis Psallidas, Microsoft, Fotis.Psallidas@microsoft.com
- Secretaries
  - Jamie Yu, Cisco, jamyu2@cisco.com
- Editors
  - David Kemp, NSA, d.kemp@cyber.nsa.gov
  - Stefan Hagen, Individual, stefan@hagen.link

Special Thanks

The following individuals have made substantial contributions to this document, not just this version of the document, and their contributions are gratefully acknowledged:

The DPS TC thanks the following individuals for their assistance in the development of this document: Kristina Podnar and the Data & Trust Alliance for their contributions of the initial schema and example applications. Duncan Sparrell for supporting the TC from the charter definition to the initial structuring of this document.

Participants

The following individuals have participated in the creation of this document and are gratefully acknowledged (given family, affiliation):

- David Kemp, NSA
- Duncan Sparrell, sFractal Consulting LLC
- Kristina Podnar, Data & Trust Alliance
- Lisa Bobbitt, Cisco
- Stefan Hagen, Individual

Appendix B. Changes From Previous Version

(This appendix does not form an integral part of this Specification and is informational.)

This is the initial draft Committee Specification.

Revision History

Revision tracking is publicly available in the version control system at https://github.com/oasis-tcs/dps/commits/main.

